

Measuring Semantic Similarity with WordNet: Path-Based vs Information Content

How similar are *car* and *automobile*? What about *dog* and *cat*, or *dog* and *car*? Humans answer this instantly, but a computer needs a formal measure. **WordNet**, a lexical database that organizes English words into a hierarchy, lets us compute such a score. In this guide, we examine two main families of WordNet-based measures and demonstrate how to implement them in Python using NLTK.

1. WordNet in a nutshell

- Words are grouped into **synsets** (sets of synonyms), each representing one *sense*, e.g. `dog.n.01` is "the domestic dog".
- Synsets are linked by **hypernym** ("is-a") relations, forming a tree-like taxonomy: `dog` → `canine` → `carnivore` → `mammal` → `animal` → ... → `entity`
- The **Lowest Common Subsumer (LCS)** of two synsets is their most specific shared ancestor. For *dog* and *cat* it is `carnivore`.

The closer two concepts are in this hierarchy, and the more specific their shared ancestor, the more similar they are.

2. Setup

First, install NLTK via pip:

```
pip install nltk
```

Download the required corpora and WordNet resources:

```
import nltk
from nltk.corpus import wordnet as wn, wordnet_ic

nltk.download("wordnet")
nltk.download("omw-1.4")
nltk.download("wordnet_ic")  # needed only for IC-based measures
```

3. Category 1: Path-based measures

These use only the **structure of the taxonomy**, meaning shortest path distances and node depths.

Measure	Idea
---------	------

Measure	Idea
Path Similarity	$1 / (\text{shortest_path_length} + 1)$
Wu-Palmer	$2 * \text{depth}(\text{LCS}) / (\text{depth}(s1) + \text{depth}(s2))$
Leacock-Chodorow	$-\log(\text{path_length} / (2 * \text{max_depth}))$

Wu-Palmer is an effective choice because it explicitly accounts for depth: two nodes sharing a *deep* (specific) ancestor are treated as significantly more similar than two nodes that only converge at a generic root ancestor.

```
dog = wn.synset("dog.n.01")
cat = wn.synset("cat.n.01")

print(dog.wup_similarity(cat))          # ~0.857
print(dog.path_similarity(cat))         # 0.2
print(dog.lowest_common_hyponyms(cat))  # [Synset('carnivore.n.01')]
print(dog.shortest_path_distance(cat))  # 4
```

Scores typically lie in the range 0 to 1. A score of 1 indicates that the synsets are identical.

4. Category 2: Information Content (IC) based measures

Path-based measures assume every edge in the taxonomy represents the same uniform semantic distance, which does not hold true across dense and sparse subtrees. IC-based measures resolve this limitation by incorporating **corpus statistics**.

Information Content

```
IC(concept) = -log P(concept)
```

$P(\text{concept})$ is the probability of seeing the concept (or any of its descendants) in a reference corpus. Rare, specific concepts (like *carnivore*) have **high IC**. Generic ones (like *entity*) have **low IC**.

Common IC measures

Measure	Formula
Resnik	$IC(\text{LCS})$

Measure	Formula
Lin	$2 * IC(LCS) / (IC(s1) + IC(s2))$
Jiang-Conrath	$1 / (IC(s1) + IC(s2) - 2 * IC(LCS))$

- Resnik only looks at the shared ancestor, making its scores unbounded and unnormalized.
- **Lin** normalizes by the IC of both synsets, yielding values strictly between 0 and 1, making it directly comparable with Wu-Palmer.

Loading IC and computing similarity

NLTK provides pre-computed IC files derived from standard reference corpora such as the Brown corpus:

```
brown_ic = wordnet_ic.ic("ic-brown.dat")

print(dog.lin_similarity(cat, brown_ic))      # ~0.877
print(dog.res_similarity(cat, brown_ic))
print(dog.jcn_similarity(cat, brown_ic))
```

Tip: You can also build your own IC file from any corpus using `wn.ic(corpus_reader, weight_senses_equally=False)`.

5. Handling words, not synsets

The measures above operate directly on **synsets**, whereas real-world input is provided as **words**. Because a word can have multiple polysemous senses (for example, *car* can denote an automobile, a train carriage, or a cable car), a standard strategy evaluates **all sense pairs** and selects the **maximum** score:

```
def best_similarity(word1, word2, measure, pos=wn.NOUN):
    best = (0.0, None, None)
    for s1 in wn.synsets(word1, pos=pos):
        for s2 in wn.synsets(word2, pos=pos):
            score = measure(s1, s2)
            if score is not None and score > best[0]: # some measures return None
                best = (score, s1, s2)
    return best
```

You can then plug in any similarity measure:

```
wup = lambda a, b: a.wup_similarity(b)
lin = lambda a, b: a.lin_similarity(b, brown_ic)

score, s1, s2 = best_similarity("car", "bicycle", wup)
```

6. Putting it together

Loop over candidate word pairs and compare both similarity metrics side by side:

```
word_pairs = [("car", "automobile"), ("car", "bicycle"),
              ("dog", "cat"), ("dog", "car"), ("gold", "egg")]

for w1, w2 in word_pairs:
    wup_score, _, _ = best_similarity(w1, w2, wup)
    lin_score, _, _ = best_similarity(w1, w2, lin)
    print(f"{w1}-{w2}: Wu-Palmer={wup_score:.3f} Lin={lin_score:.3f}")
```

Sample output

Pair	Wu-Palmer	Lin
car - automobile	1.000	1.000
coast - shore	0.909	0.963
journey - voyage	0.952	0.778
dog - cat	0.857	0.877
car - bicycle	0.800	0.766
dog - car	0.667	0.381
gold - egg	0.400	0.323

7. Observations

- **Synonyms score 1.0** in both methods since they share identical synsets.

- **Related concepts** (e.g. dog/cat, car/bicycle) score high, while unrelated concepts (e.g. gold/egg) receive significantly lower scores.
- **IC-based scores spread out more.** For dog - car, Wu-Palmer yields 0.667 while Lin drops sharply to 0.381. Because they only share a high-level, generic ancestor with low IC, Lin applies a stricter penalty. Path-based measures tend to be overly optimistic in shallow hierarchies.
- **Sense selection matters.** Taking the maximum score across all sense pairs can occasionally select unexpected synsets (e.g. matching dog to car via pawl.n.01, a mechanical catch mechanism). For higher precision, restrict comparisons to the most frequent sense (`wn.synsets(word)[0]`) or apply contextual word sense disambiguation (WSD).

8. Limitations and tips

- **Same Part-of-Speech Constraint:** Measures require both concepts to share the same part of speech (nouns with nouns, verbs with verbs). Path and Wu-Palmer depend on a shared taxonomic root, and IC formulations are defined only for nouns and verbs.
- **No Hierarchy for Adjectives/Adverbs:** Adjectives and adverbs lack hypernym trees in WordNet, meaning these measures cannot be directly evaluated on them.
- **Corpus Sensitivity:** Information Content scores depend heavily on the reference corpus used. Frequency shifts across domains will alter similarity values.
- **Selecting the Right Measure:** Choose Wu-Palmer or Path similarity for a lightweight, corpus-independent baseline. Choose Lin or Jiang-Conrath when you need fine-grained discrimination based on empirical concept specificity.

9. Summary

Step	Action / Implementation
1	Install NLTK and download wordnet, omw-1.4, and wordnet_ic
2	Load a pre-compiled Information Content file (e.g. ic-brown.dat)
3	Retrieve candidate synsets for each word using <code>wn.synsets(word, pos=...)</code>
4	Score each sense pair using a path-based measure (<code>wup_similarity</code>) and an IC-based measure (<code>lin_similarity</code>)
5	Identify the maximum score per pair and analyze relative discriminative differences